Step 1: Clone the GitHub Repository

bash

```
git clone https://github.com
cd YOUR_REPO_NAME
```

Step 2: Build the Docker Image

They must compile the application blueprint into a runnable Docker image.

bash

```
docker build -t your_dockerhub_username/python-web-app:latest .
```

Use code with caution.

Step 3: Login and Push to Docker Hub

To make the image accessible to AWS, you must authenticate and upload it to Docker Hub.

bash

```
# Authenticate using Docker Hub credentials
docker login

# Upload the image to the public repository
docker push your_dockerhub_username/python-web-app:latest
```

Step 4: Provision the AWS Infrastructure with Terraform

To deploy this image onto AWS EC2 automatically, you will use Terraform.

1. **Create the infrastructure files:** Have them create a directory named `terraform/` and save the following configuration as `main.tf`.

hcl

```hcl
provider "aws" {
  region = "us-east-1" # Change to their preferred AWS region
}

# 1. Create a Security Group to allow web traffic
resource "aws_security_group" "web_sg" {
  name        = "app-security-group"
  description = "Allow HTTP and SSH traffic"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # Open for setup (restrict in production)
  }

  ingress {
    from_port   = 5000
    to_port     = 5000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # Port where the Python app runs
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# 2. Launch the EC2 Instance and automatically run the Docker Container
resource "aws_instance" "web_server" {
  ami           = "ami-0c7217cdde317cfec" # Standard Ubuntu 22.04 LTS AMI
in us-east-1
  instance_type = "t2.micro"             # Free tier eligible

  vpc_security_group_ids = [aws_security_group.web_sg.id]

  # User Data script runs automatically when the machine starts up
  user_data = <<-EOF
              #!/bin/bash
              sudo apt-get update -y

              # Install Docker
```

```
              sudo apt-get install -y apt-transport-https ca-certificates
curl gnupg lsb-release
              curl -fsSL https://docker.com | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg
              echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://docker.com $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
              sudo apt-get update -y
              sudo apt-get install -y docker-ce docker-ce-cli
containerd.io

              # Start Docker service
              sudo systemctl start docker
              sudo systemctl enable docker

              # Pull and run the container from Docker Hub
              sudo docker run -d -p 5000:5000
your_dockerhub_username/python-web-app:latest
              EOF

  tags = {
    Name = "PythonAppServer"
  }
}

# 3. Output the public URL to the terminal screen
output "public_ip" {
  value       = "http://${aws_instance.web_server.public_ip}:5000"
  description = "The public URL of your live Python web application"
}
```

Use code with caution.

2. **Execute the Terraform commands:** You runs these commands inside the `terraform/` directory to build the cloud infrastructure:

bash

```
# Initialize the project and download AWS providers
terraform init

# Preview the infrastructure additions
terraform plan

# Deploy the infrastructure live to AWS
```

```
terraform apply -auto-approve
```

Use code with caution.

Step 5: Access the Live Application

Once the `terraform apply` command finishes successfully, it will print out the target URL:

text

```
Outputs:
public_ip = "http://54.210.45.12:5000"
```